

EXAMEN CONVOCATORIA ORDINARIA PMDM

La prueba consiste en añadir una serie de nuevas funcionalidades a una aplicación Android que aplica la arquitectura **Clean Architecture** y que persiste los datos en una base de datos local mediante Room.

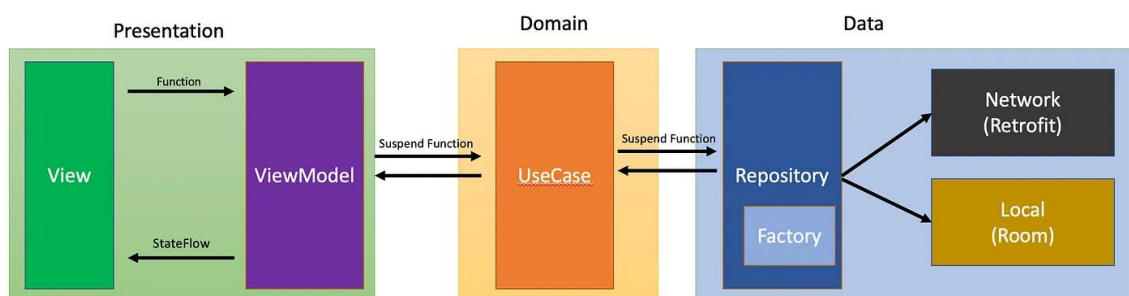
La entrega de la app se debe realizar en la plataforma Aules, mediante la subida del proyecto Android Studio comprimido en zip.

La aplicación consiste en un listado de citas acerca del mundo de la informática, con su texto y su autor. Cada cita del listado se puede marcar como favorita o verla en una vista de detalle, donde se podrá, además de añadirla a favoritas, compartirla. La propia pantalla de listado permite las operaciones de filtrado de citas por texto (cita o autor), ver solamente las citas favoritas y ver una cita aleatoria en la vista de detalle.

La aplicación es funcional tal como se entrega a falta de añadir los dos nuevos casos de uso: añadir nuevas citas y eliminarlas.

Tu labor consiste en aplicar los conocimientos adquiridos durante el curso **para proporcionar a la aplicación la nueva funcionalidad que permita añadir y eliminar citas**, persistir la información y mantener el estado de la UI cuando cambie la configuración del dispositivo (por ejemplo, una rotación) y todo ello aplicando las buenas prácticas de diseño definidas en Clean Architecture:

- Separación de responsabilidades en tres capas:
 - **Data:** Persistencia de la información
 - **Domain:** Lógica de la aplicación
 - **Presentation:** Interacción con el usuario y mostrar información
- Domain es una capa que no depende de ninguna otra (es el núcleo central) ni depende de ningún artefacto de Android (es código Kotlin puro)
- Data y Presentation son capas de infraestructura y si son dependientes de Android y de Dominio, pero no se conocen entre si



Instrucciones iniciales

Descarga de aules el fichero zip con la plantilla inicial del proyecto y el pdf con el gui3n.

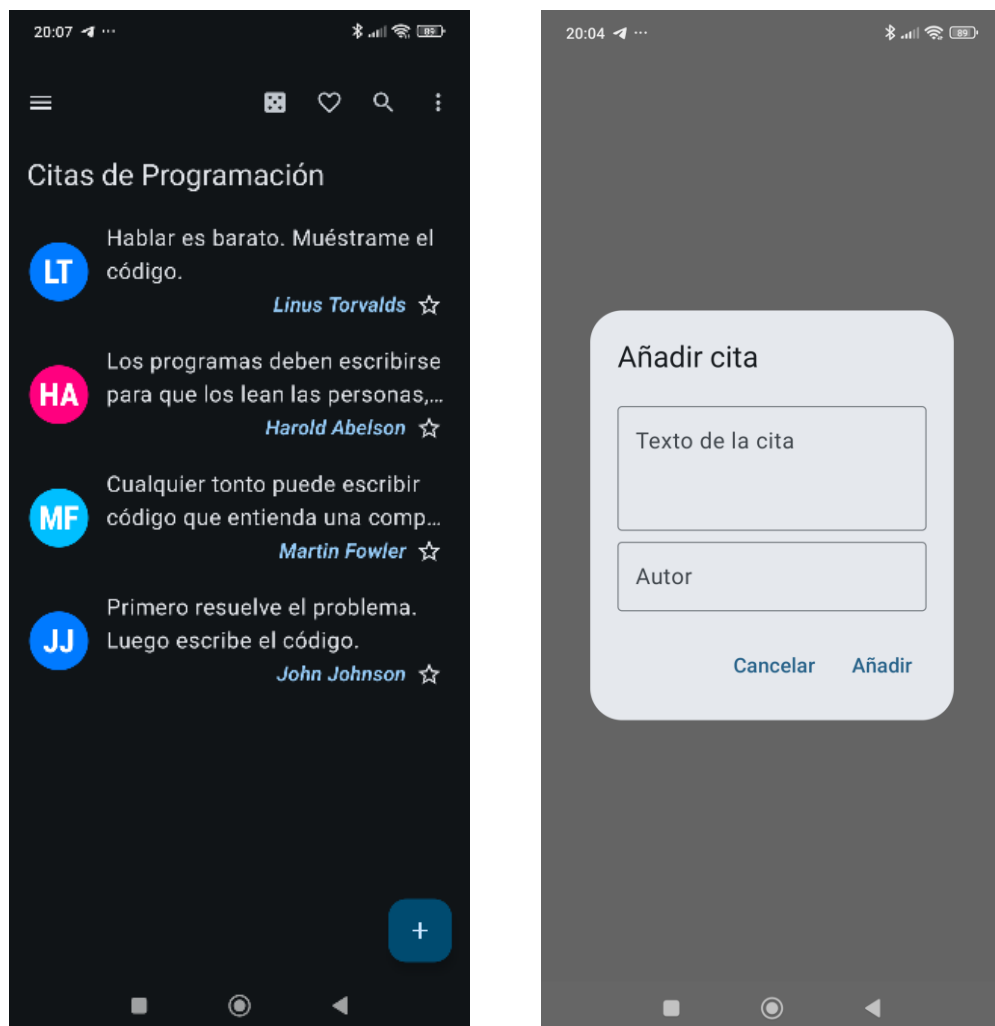
Descomprime el archivo zip en una carpeta y 3brela con Android Studio.

Tu trabajo consiste en ampliar la funcionalidad de la aplicaci3n con dos nuevos casos de uso:

- A3nadir nueva cita
- Eliminar cita

A3nadir cita

Para a3nadir una nueva cita se debe poner a disposici3n del usuario un Floating Action Button con el icono de + para que si lo pulsa se abra un Di3logo. En el di3logo se debe pedir al usuario que introduzca el texto de la cita y el autor mediante dos textfields y que a3nada o cancele mediante TextButtons para cancelar o a3nadir.

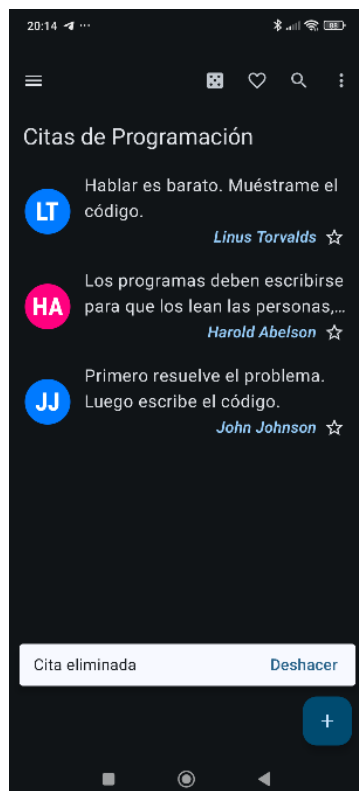


Eliminar cita

Cuando el usuario realice el gesto swipe de derecha a izquierda de un ítem de la lista de citas desencadenará el procedimiento para eliminar la cita de la lista.



Al eliminarla, se debería notificar al usuario de la posibilidad de deshacer la operación mediante un snackbar. Si el usuario pulsa el Deshacer se debe restaurar la cita en la lista.



Información adicional

Para que el item de lista de citas permita el gesto de swipe que dispare la acción de eliminación se proporciona un componente composable **SwipeableQuoteListItem** lista para usar dentro del paquete `components` de la pantalla de lista de citas. Este componente envuelve al `QuoteListItem` original usándolo como base.

Debes aplicar en todo momento el patrón MVI en la capa de presentación, y mantener la separación de responsabilidades aplicando Clean Architecture.

Por tanto, añade el código que sea necesario en `viewModels`, `DAOs`, pantallas, casos de uso, repositorios, módulos de Koin, etc.

Para añadir nueva funcionalidad en MVI debes añadir nuevas acciones a la lista de acciones y nuevos eventos si son eventos de un solo uso y gestionarlos en el `viewmodel` y en la pantalla de `route` que recibe el `viewmodel`.

El estado del `snackbar` de una pantalla debe estar en la función composable `_Route`, (la que recibe en `viewmodel` como parámetro)

Entrega

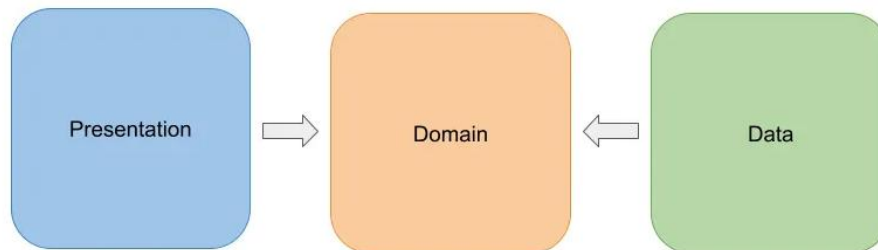
Comprime el proyecto en un zip y súbelo a aules.

Menu → File → Export → Export to zip file...

Anexo - Explicación estructura del código de la app

Capa de dominio

La capa de dominio en **Clean Architecture** es la capa nuclear, no depende de ninguna otra capa (en otras palabras, no conoce ninguna clase o código de las otras dos capas).

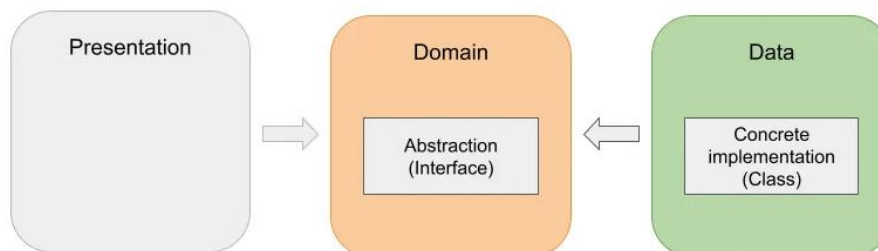


Modelos de dominio

En la capa de dominio se definen los modelos de datos de dominio. En este caso la clase **Quote** que ya la tienes disponible

Interfaces de repositorios

También se define la interface de repositorio que actúa como contrato de la funcionalidad que debe implementar la clase implementadora del repositorio que se define en la capa de datos. Se aplica el principio e inversión de dependencias para que Data dependa de Domain.



Localiza la interface de repositorio para las citas **QuoteRepository** en el subpaquete “repository” dentro del paquete “domain”.

Los métodos de la interface repositorio son los siguientes:

getQuotes() → Devuelve un flow que emite una lista de citas (modelo del dominio)

toggleFavorite(quote:Quote) → alterna el valor de favorita de la cita

searchQuotes(query: String) → Devuelve un flow que emite una lista de citas filtradas

getQuoteById(id: Int) → Devuelve un flow que emite la cita que corresponde con el id

getRandomQuote() → Devuelve un flow que emite una cita aleatoria

Recuerda que toda función que no devuelva un Flow debe ser declarada como una función suspendida.

Tendrás que ampliar la interface para que permita operaciones de insertar cita y eliminar cita.

Casos de uso

En el paquete dominio, el subpaquete **usecase** contiene todas las clases que modelan los casos de uso de la app.

ToggleFavoriteUseCase

Este caso de uso permite al usuario alternar que una determinada cita sea considerada favorita o no. Delega el cambio de valor de favorito de la cita en un objeto **QuoteRepository** del cual depende.

GetRandomQuoteUseCase

Este caso de uso permite obtener los datos de una cita aleatoria de entre todas las citas que están almacenadas en el sistema de persistencia. Para obtenerla hace uso de los servicios que le presta un **QuoteRepository** del cual, por tanto, depende.

GetFilteredQuotesUseCase

El cometido de este caso de uso es obtener, a partir de un texto utilizado para filtrar citas, una lista de citas filtradas. En otras palabras, las citas cuyo texto o autor contiene el texto proporcionado para filtrar. Simplemente delega en un **QuoteRepository**.

Al invocar el caso de uso hay que proporcionarle la query que contiene el texto para filtrar las citas.

GetQuoteUseCase

Este caso de uso permite obtener la cita cuya propiedad id se corresponde con el identificador proporcionado para su identificación. De nuevo se delega en un **QuoteRepository**.

Este caso de uso necesita que se le proporcione al invocarlo el identificador de la cita a recuperar del almacén de datos.

Los casos de uso se deben crear como clases que contienen un único método público que defina el operador **invoke**. De esta manera se facilita la llamada al caso de uso puesto que si, por ejemplo, tenemos una referencia al caso de uso `GetRandomQuoteUseCase` en la variable `getRandomQuoteUseCase`, podemos llamar al método `invoke` de dos maneras:

- `getRandomQuoteUseCase()` → forma sencilla e implícita
- `getRandomQuoteUseCase.invoke()`

Recuerda:

- El método `invoke` debe ser declarado como suspendido a no ser que devuelva un Flow.

Capa de datos

En el paquete **data** colocamos todo el código cuya responsabilidad es la de persistir la información que maneja la aplicación. En este caso, las citas se persisten en una base de datos local SQLite mediante el ORM Room.

Room es un ORM, y trabaja con modelos de datos de la capa de datos para persistir en un base de datos relacional, más conocidos como entidades.

Entidad

Localiza la entidad **QuoteEntity** en el subpaquete **entity** y comprueba que esta mapeada mediante anotaciones para que se guarden sus instancias como filas en una tabla con nombre **quotes**.

La clave primaria es autogenerada

La tabla **quotes** cuenta con los atributos: ID, TEXT, AUTHOR, IS_FAVORITE

Dao

El DAO es un Data Access Object y su responsabilidad es transformar llamadas a métodos en la ejecución de una consulta SQL que realiza una acción en la base de datos que puede ser de inserción, actualización, borrado o selección: CRUD

Dentro de **data**, en la carpeta **local** se define el código del QuoteDao que realice las siguientes operaciones:

- **getAllQuotes()** → Devuelve un Flow que emite todas las citas de la BD
- **getQuoteById(id:Int)** → Devuelve un Flow que emite la cita con el id proporcionado
- **deleteAllQuotes()** → Borra todas las citas de la BD
- **getQuotesCount()** → Devuelve el número de total de citas en la BD
- **searchQuotes(query:String)** → Devuelve un flow que emite todas las citas que pasan el filtro indicado por query, todas las citas que contengan el texto del parámetro query en el texto de la cita o en el nombre del autor
- **getRandomQuote()** → Devuelve un Flow que emite una cita aleatoria
- **updateQuote** y **updateQuotes** para actualizar una cita o una lista
- **toggleFavorite(quoteId:Int)** → alterna el valor del campo is_favorite en la cita con id igual al proporcionado por el parámetro quoteId

Tendrás que añadir métodos para insertar y eliminar una cita en la base de datos.

RECUERDA:

- Las funciones que devuelven flows pueden ser funciones normales

- Las funciones de modificación de datos y las que no devuelven flows deben ser funciones suspendidas

Base de datos

Localiza la clase **AppDatabase** en la carpeta **local** que maneja una base de datos con un esquema que contiene una tabla para la entidad **QuoteEntity**. La clase declara un método **quoteDao** que devuelva una referencia a un objeto de tipo **QuoteDao** para realizar operaciones con las citas de la base de datos.

Para iniciar una instancia de base de datos se necesita una referencia al contexto de la aplicación. Por este motivo, se definen los siguientes dos métodos de extensión de **Context** dentro del mismo fichero donde se declara la clase AppDatabase.

El método **getDatabase** devuelve una base de datos que persiste la información de manera permanente ya que usa un fichero como almacén de la base de datos.

```
fun Context.getDatabase(): AppDatabase {
    return Room.databaseBuilder(
        this,
        AppDatabase::class.java,
        "quotes.db"
    ).build()
}
```

La función **getInMemoryDatabase** devuelve una base de datos que trabaja directamente en memoria y por tanto, las acciones no son persistentes.

```
fun Context.getInMemoryDatabase(): AppDatabase {
    return Room.inMemoryDatabaseBuilder(
        this,
        AppDatabase::class.java
    ).build()
}
```

Podrás elegir una u otra en la declaración de la dependencia en el módulo de Koin.

Mappers

En la capa de datos se implementa la interface del repositorio cuyos métodos trabajan con referencias de tipo **Quote** (modelo del dominio) y no **QuoteEntity** que es una modelo de datos (y que no debe ser expuesto al resto de capas). Por tanto, necesitamos convertir un QuoteEntity a un Quote equivalente y viceversa. Esta tarea la realizan los mappers, funciones que convierten un objeto del modelo de datos en un objeto del modelo de dominio y a la inversa.

- Para convertir un QuoteEntity en un Quote se define una función de extensión de QuoteEntity llamada **toDomain()**
- Para convertir un Quote en un QuoteEntity se define una función de extensión de Quote y llamada **toEntity()**

Estas funciones están definidas en un archivo **QuoteMappers.kt** en la carpeta **local** de **data**.

Repositorio (implementación)

En la capa de datos se implementa la interface de repositorio declarada en la capa de dominio.

Localiza la subcarpeta **repository** dentro de data y luego dentro de esta una clase **QuoteRepositoryImpl** que implementa la interface **QuoteRepository**.

La clase QuoteRepositoryImpl necesita una referencia a un **QuoteDao** para poder llevar a cabo su trabajo, que básicamente consiste en delegar en el QuoteDao las acciones y mapear los resultados que este le entrega, ya que el QuoteDao utiliza objetos QuoteEntity.

Por tanto, se apoyan en los métodos mapeadores **toDomain** y **toEntity** para transformar tanto listas como objetos QuoteEntity a Quote y viceversa cuando es necesario.

Capa de presentación

Reubicación de elementos de la UI en Compose

El paquete **presentation** es para la capa de presentación. Dentro tenemos 3 subpaquetes:

- **Components** para los componentes composables no específicos de una pantalla puesto que se reutilizan en más de una pantalla.
- **Screens** para las pantallas de la UI en Compose
- **Utils** para funciones de utilidad de la UI

ViewModels

Existe un viewModel para la pantalla de **QuoteDetails** y otro para la pantalla de **QuoteList**, con nombre igual al de la pantalla y sufijo ViewModel.

Para la capa de presentación se sigue el **patrón MVI**, y por tanto, se definen las interfaces que contiene las acciones que puede realizar el usuario cuando interactúa con la pantalla.

En el viewModel se declara un método **onAction** que recibe una referencia a la acción efectuada por el usuario, comprueba el tipo concreto de la acción y llama a una función privada creada para manejarla.

Para el caso de la pantalla de detalle las acciones son las siguientes (QuoteDetailsAction):

- **OnFavoriteClick** (el usuario ha tocado el chip de favorito)
- **OnShareClick** (el usuario ha tocado el chip de compartir)

Para el caso de la pantalla de lista de citas las acciones son las siguientes (QuoteListAction):

- **OnRandomQuote** (el usuario ha tocado el botón del dado para ver una cita aleatoria)
- **OnAbout** (para navegar a la pantalla de Acerca de)
- **OnToggleFilterFavorites** (el usuario ha tocado el icono de favoritos de la topbar)
- **OnToggleSearchBar** (el usuario ha tocado el icono lupa que muestra / oculta la barra de búsqueda)
- **OnToggleFavorite** (el usuario ha tocado el icono favorito de una cita de lista)
- **OnQueryChange** (el usuario ha modificado la caja de texto de búsqueda)
- **OnQuoteSelected** (el usuario ha seleccionado una cita al tocar una cita de la lista)

Añade la acciones para añadir y eliminar citas

Eventos MVI

Localiza el archivo `ObserveAsEvents.kt` dentro del paquete `presentación`, que se usa para procesar los eventos de un solo uso.

```

@Composable
fun <T> Flow<T>.ObserveAsEvents(onEvent: suspend (T) -> Unit) {

    val lifecycleOwner = LocalLifecycleOwner.current

    LaunchedEffect(lifecycleOwner) {

        lifecycleOwner.lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
            withContext(Dispatchers.Main.immediate) {
                collect { event -> onEvent(event) }
            }
        }
    }
}

```

Para la pantalla de lista de citas

Cuando el usuario pulsa en el icono del dado se navega a la pantalla de detalle para mostrar una cita aleatoria. Es decir, en respuesta a la acción `OnRandomQuote` hay que obtener una cita aleatoria y navegar a la pantalla de detalle para mostrarla.

Maneja esto mediante un evento de un solo uso **OnNavigateToQueryDetails**.

Define este evento como una implementación de la interface **QuoteListEvent**.

Crea un `Channel< QuoteListEvent>` dentro del `viewModel` para la lista de citas.

Cuando manejes la acción `OnRandomQuote` obtén la cita aleatoria mediante el caso de uso `GetRandomQuoteUseCase` y a continuación envía el evento por el canal para que pueda ser observado en la función `QuoteListRoute` usando la función `ObserveAsEvents`.

Para la pantalla de detalle de cita

Desde esta pantalla se puede compartir la cita. Para compartir la cita dispones del método de extensión **share** definido en el archivo `ShareUtils.kt`. Este método necesita que se le proporcione una referencia al contexto y por tanto, no se puede utilizar dentro de una función del `viewModel`.

Por tanto, el `viewModel` debe enviar un evento a la función composable `_Route` de la UI para que esta lo observe (recolecte) y desde ahí llamar al método `share` pasándole el contexto que podemos obtener en `Compose` mediante `LocalContext.current`.

Define una interface para los eventos de UI de la pantalla de detalle y llámalo **QuoteDetailsEvent**.

Define una clase de evento **OnShare** que recibe un parámetro de tipo `Quote` e implementa `QuoteDetailsEvent`.

En el `viewModel` para la vista de detalle crea un canal para enviar eventos de tipo `QuoteDetailsEvent`.

Envía un evento `OnShare` por el canal como respuesta a la acción `OnShare` recibida en el `viewModel` y generada por el usuario al tocar el botón chip de compartir de la pantalla.

Estado de la UI

El estado de la UI se mantiene en los viewModels mediante StateFlows.

Para la pantalla de lista de citas crea una clase QuoteListUiState que será el tipo de elementos del StateFlow y contendrá todos los elementos necesarios para determinar el estado de la pantalla de lista de cita:

- **query** → el texto de la barra de búsqueda (para filtrar citas)
- **quotes** → una lista de citas (filtrada)
- **showSearchBar** → indica si se muestra o no la barra de búsqueda
- **showFilterFavorites** → indica si se muestran todas las citas o solamente las citas favoritas

Para la pantalla de detalle, dado que el estado consiste simplemente en la cita a mostrar no hace falta que crees una clase QuoteDetailsUiState, puedes usar directamente un StateFlow que emita un Quote directamente.

El viewModel de la pantalla de detalles se puede apoyar en el caso de uso GetQuoteUseCase que devuelve un Flow<Quote> y convertirlo en un StateFlow mediante la función **stateln**

Filtrado de citas

Para filtrar las citas debes observar (recolectar) el StateFlow en el propio viewModel para detectar cuando se emite QuoteListUiState que tenga un valor del atributo **query** distinto al de la emisión anterior y a continuación usarlo para pedirle al caso de uso GetFilteredQuotesUseCase (dependencia del viewModel) que recupere las citas que pasan el filtro indicado mediante el valor del parámetro query. Este devolverá un Flow<List<Quotes>> por lo que tenemos que aplicar la función **flatMapLatest** para que el flow de flows Flow<Flow<List<Quotes>>> se aplane a un Flow<List<Quotes>> y ahora, por cada elemento (emisión) actualizar el uiState, concretamente su propiedad quotes.

Recuerda:

Para actualizar un MutableStateFlow (generar una nueva emisión) se utilizr preferiblemente la función **update**.

Por ejemplo, para actualizar el valor de la propiedad query de un QuoteListUiState referenciado por la variable _uiState:

```
_uiState.update { it.copy(query = action.query) }
```

Composables _Route

Por cada pantalla a la que se puede navegar y que use un viewModel debes crear una funcion composable con nombre de la pantalla y sufijo Route que reciba el viewModel y:

- Convierta el estado de la UI en un State observable de Compose
- Procese los eventos

- Llame a la función Composable que muestra la pantalla

Las funciones `_Route` reciben una referencia al `viewModel`.

Además, pueden declarar otros parámetros para recibir:

- Callbacks para navegación. Por ejemplo, un `onBack` para ir a la pantalla anterior, `onAbout` para ir a la pantalla de About, `onNavigateToDetails` para ir a la pantalla de detalles, ...
- El ID de una cita para identificar la cita que se quiere mostrar (en el caso de la pantalla de detalle)

Composable QuoteDetailsRoute

La función para la pantalla de detalles de cita **QuoteDetailsRoute** debería observar los eventos de un solo uso **QuoteDetailsEvent**, en concreto el **OnShare** para cada vez que se envíe un evento de este tipo compartir la cita.

Debe recibir además del `viewModel`:

- **quoteId** → El identificador de la cita
- **onBack** → callback para la navegación a la pantalla anterior en la pila de `backStack`

Cuando se llama a `QuoteDetailsRoute` desde la función `NavigationRoot` se proporcionan los argumentos para estos parámetros.

Llama a la función composable de la pantalla de detalle de la cita.

Composable QuoteListRoute

La función para la pantalla de lista de citas **QuoteListRoute** tiene que observar los eventos de un solo uso **QuoteListEvent**, en concreto el **OnNavigateToQuoteDetails** para navegar a la pantalla de detalles cuando se observe un evento de este tipo.

Debe recibir, además del `viewModel`, dos parámetros con los callbacks para navegar a la pantalla de About y a la pantalla de detalles:

- **onNavigateToQuoteDetails**: `(Int) -> Unit`
- **onNavigateToAbout**: `() -> Unit`

Cuando se llama a `QuoteListRoute` desde la función `NavigationRoot` se proporcionan los argumentos para estos dos parámetros indicando que claves apilar en la pila de `backStack` para navegar a dichas pantallas cuando sea necesario.

También debería declarar una función (o una variable de tipo función) que reciba una referencia a un `QuoteListAction` y si se trata de un:

- **OnAbout** → que invocará a `onNavigateToAbout`
- **OnQuoteSelected** → que invocará a `onNavigateToQuoteDetails` proporcionando el identificador de la cita
- El resto de las acciones las delega llamando al método `onAction` del `viewModel` pasándole la acción.

Convierte el `StateFlow` del estado de la UI proporcionado por el `viewModel` en un estado de `Compose` mediante la función **collectAsStateWithLifecycle()**

Llama a la función composable de la pantalla de lista de citas.

Composables de pantalla MVI

Son las funciones composables que determinan como se debe representar la UI de una pantalla en base a un estado. Estas funciones no reciben el viewModel como parámetro, sino las funciones correspondientes que tienen el sufijo Route. Esto las hace previsualizables en el IDE.

Estas funciones reciben:

- **QuoteListScreen**
 - o **uiState** de tipo **QuoteListUiState**
 - o **onAction** de tipo función que recibe un **QuoteListAction** y no devuelve nada.
- **QuoteDetailsScreen**
 - o **quote** de tipo **Quote?**
 - o **onAction** de tipo función que recibe un **QuoteDetailsAction** y no devuelve nada.
 - o **onBack** de tipo función que no recibe nada y no devuelve nada.

Navegación entre pantallas

La navegación la vamos a situar en la subcarpeta “navigation”

Mueve a esta carpeta el archivo NavigatioRoot.kt.

Modifica las entradas del DSL del entryProvider para que el contenido asociado con las NavKeys llame a las funciones composables QuoteDetailsRoute en lugar de QuoteDetailsScreen y QuoteListRoute en lugar de QuoteListScreen.

Recuerda:

- Para que el ciclo de vida de un viewModel esté ligado a la permanencia de una entrada en la pila de backStack es necesario proporcionar los entry decorators necesarios en la llamada a la función NavDisplay

Inyección de dependencias

Se utiliza el patrón DI (dependency injection) en el desarrollo de la app. Concretamente se usa Koin como inyector.

Dentro del paquete **di** se ubica el archivo **AppModule.kt** que contiene la definición de una variable global **appModule** con la definición de un módulo de Koin que contiene declaraciones de las dependencias a inyectar en la aplicación.

Declara una dependencia de tipo viewModel por cada viewModel y se usa la función **koinViewModel** en aquellas funciones donde se quiera inyectar el viewModel como parámetro de entrada.

En la declaración de la dependencia del viewModel para la pantalla de detalles se ha tenido en cuenta que el **quoteId** se inyecta como parámetro dinámico (dado que su valor se recibe desde el key de navegación)

Para que Koin realice su trabajo en tiempo de ejecución se debe iniciar Koin desde el método **onCreate** de la subclase **QuoteApp** que está configurada como clase de la aplicación en el manifest.xml.

Este código se utiliza dentro del DSL del módulo de Koin para definir la dependencia de la base de datos:

```
single {
    androidContext().getInMemoryDatabase().also { database ->
        CoroutineScope(Dispatchers.IO).launch {
            with(database.quoteDao()) {
                if (getQuotesCount() == 0)
                    insertQuotes(QuoteProvider.programmingQuotes.map {
                        it.toEntity()
                    })
            }
        }
    }
}
```

Este código, además de crear la base de datos, carga en la tabla quotes de la base de datos las citas que hay definidas en el objeto QuoteProvider si la tabla no tiene citas.

Puedes cambiar la llamada a getInMemoryDatabase por getDatabase cuando quieras que la aplicación persista los datos cuando se cierre la app.